Site Alsimat — Package de reprise

Ce package contient tout le nécessaire pour héberger, maintenir et faire évoluer le site **Alsimat** (concessionnaire de véhicules électriques : tracteurs, nacelles, tricycles…) en dehors de sa plateforme d'origine.

Dossier	Contenu
`code/`	Code source complet de l'application (Next.js 14, TypeScript, Prisma) + `.env.example`
`database/`	Export complet de la base PostgreSQL (`alsimat_database.sql`, schéma + données) et copie du schéma Prisma (`schema.prisma`)
`assets/`	Logo officiel (`alsimat_logo_official.png`) et copie de toutes les images actuellement utilisées par le site (`images_existantes/` + `correspondance_urls.csv`)
`README.md` / `README.pdf`	Ce document

---

1. Présentation du site

Application **Next.js 14 (App Router)** en TypeScript, avec :

- **Prisma** (ORM) + **PostgreSQL** pour les données ;
- **NextAuth** (fournisseur « credentials » : email + mot de passe hashé bcrypt, sessions JWT) pour l'espace administrateur ;
- **Tailwind CSS** + composants **shadcn/ui** (Radix UI) pour l'interface ;
- **AWS SDK S3** (URLs présignées) pour l'upload des images.

Fonctionnalités publiques

Page	URL	Description
Accueil	`/`	Bandeau principal, gammes, promotions, arguments, appel à l'action — contenus éditables depuis l'admin (CMS)
Gammes / sous-gammes	`/gammes/[slug]`	Catalogue par catégorie, avec hiérarchie catégories → sous-catégories
Véhicules	`/vehicules/[slug]`	Fiche détaillée (caractéristiques, galerie, prix, demande de devis). Un véhicule peut appartenir à plusieurs catégories
Réalisations	`/realisations`	Galerie de projets clients
Occasion	`/occasion`, `/occasion/[slug]`	Véhicules d'occasion (prix, année, heures/kilométrage, état, statut

Page	URL	Description
		vendu)
Promotions	`/promotions`	Offres en cours (remise, badge, bouton d'action)
Configurateur	`/configurateur`	Configuration d'un modèle (groupes d'options, prix de base + suppléments) avec envoi d'une demande de devis
Demandes de devis	(formulaires)	Depuis les fiches véhicules, le configurateur, l'occasion… enregistrées en base
SAV	`/sav`	Formulaire de ticket SAV (n° de série, description, photos jointes uploadées sur S3)
Contact	`/contact`	Formulaire de contact enregistré en base
À propos	`/a-propos`	Page éditable depuis l'admin (CMS)
SEO	`/sitemap.xml`, `/robots.txt`	Générés automatiquement

Espace administrateur (`/admin`)

Accès protégé (`/admin/login`) — réservé aux utilisateurs ayant le rôle `admin`.

- **Tableau de bord** : statistiques (véhicules, demandes, tickets…).
- **Véhicules** : création/édition/suppression, multi-catégories, images, mise en avant.
- **Catégories** : gammes et sous-gammes (champ parent), ordre d'affichage, images.
- **Occasion**, **Promotions**, **Réalisations** : gestion complète avec images.
- **Configurateur** : modèles, groupes d'options, options et suppléments de prix.
- **Demandes de devis**, **Tickets SAV**, **Contacts** : consultation et suivi (statuts).
- **Contenu du site** : CMS des pages Accueil et À propos.
- **Paramètres** : interrupteurs de visibilité des rubriques Réalisations, Occasion, Promotions et Configurateur (stockés dans la table `SiteSetting`).

Modèles de données (Prisma)

`Category`, `Vehicle`, `VehicleCategory` (liaison multi-catégories), `Project` (réalisations), `UsedVehicle` (occasion), `Promotion`, `ConfiguratorModel` / `ConfigGroup` / `ConfigOption`, `QuoteRequest`, `SavTicket`, `ContactSubmission`, `User`, `SiteSetting`. Voir `database/schema.prisma`.

---

2. Prérequis

- **Node.js 18 ou plus** (testé avec Node 22).
- **Yarn** : le projet utilise Yarn 4 (déclaré dans `package.json` via `packageManager`). Activer avec `corepack enable` (fourni avec Node). `npm install` fonctionne aussi mais n'utilisera pas le `yarn.lock`.
- **PostgreSQL 13 ou plus** (base locale, VPS, ou service managé : Neon, Supabase, Scaleway, OVH, AWS RDS…), avec le client `psql`.

- Un **bucket S3 ou compatible S3** pour les nouveaux uploads d'images.

---

3. Installation pas à pas

```
# 1. Se placer dans le code
cd alsimat_site/code

# 2. Installer les dépendances (génère aussi le client Prisma via "postinstall")
corepack enable
yarn install

# 3. Créer le fichier d'environnement puis le compléter
cp .env.example .env
#   -> éditer .env et remplacer chaque <A_RENSEIGNER> (voir section 5)

# 4. Créer une base vide et y restaurer l'export
createdb alsimat                        # ou via l'interface de votre hébergeur
psql "postgresql://UTILISATEUR:MDP@HOTE:5432/alsimat" -f ../database/alsimat_database.sql

# 5. Générer le client Prisma (si non fait automatiquement)
yarn prisma generate

# 6. Lancer en développement (http://localhost:3000)
yarn dev

# 7. Production
yarn build && yarn start
```

Remarques :

- La base exportée contient déjà toutes les tables et données : **ne pas lancer `prisma migrate`** ni le seed sur cette base. Pour une évolution du schéma, modifier `prisma/schema.prisma` puis utiliser `yarn prisma db push` (ou mettre en place `prisma migrate` en partant de l'état actuel).
- L'export a été produit par `pg_dump` 17 ; il a été testé avec succès en restauration et rendu compatible avec les versions antérieures de PostgreSQL (seule la ligne `transaction_timeout`, propre à PG 17, est commentée).
- Le script `yarn prisma db seed` (`scripts/seed.ts`) remplit une base **vide** avec un jeu de données initial ; il ne doit pas être utilisé sur la base restaurée (il réécrirait certaines fiches).

---

4. Options d'hébergement

Option	Points clés
Vercel (recommandé pour Next.js)	Importer le dossier `code/` depuis un dépôt Git (GitHub/GitLab). Renseigner les variables d'environnement dans le tableau de bord. Commande de build `yarn build`. Base PostgreSQL externe (Neon, Supabase…).
Netlify / **Render** / **Railway**	Fonctionnement similaire (support Next.js natif ou via adaptateur). Render et Railway peuvent aussi héberger

Option	Points clés
	PostgreSQL.
VPS (OVH, Scaleway, Hetzner…)	Installer Node 18+, PostgreSQL, puis `yarn build && yarn start` derrière un proxy **Nginx** (HTTPS via Let's Encrypt), processus géré par **PM2** ou systemd.
Docker	Définir `NEXT_OUTPUT_MODE=standalone` au build pour obtenir un serveur autonome (`.next/standalone/server.js`) à copier dans une image `node:20-alpine` avec `public/` et `.next/static/`.

Dans tous les cas : définir `NEXTAUTH_URL` sur l'URL publique finale et pointer le nom de domaine vers le nouvel hébergement.

---

5. Variables d'environnement

Toutes sont listées et commentées dans `code/.env.example`.

Variable	Obligatoire	Rôle
`DATABASE_URL`	Oui	Connexion PostgreSQL utilisée par Prisma
`NEXTAUTH_SECRET`	Oui	Clé secrète de signature des sessions admin (`openssl rand -base64 32`)
`NEXTAUTH_URL`	Oui (prod)	URL publique du site, ex. `https://www.alsimat.fr`
`AWS_REGION`	Pour les uploads	Région du bucket (ex. `eu-west-3`)
`AWS_BUCKET_NAME`	Pour les uploads	Nom du bucket
`AWS_FOLDER_PREFIX`	Non	Préfixe/dossier ajouté devant les fichiers (ex. `alsimat/`)
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`	Pour les uploads	Clés d'un utilisateur IAM ayant les droits `PutObject`, `GetObject`, `DeleteObject` sur le bucket (facultatives si l'hébergeur fournit un rôle IAM ou un `AWS_PROFILE`)
`AWS_PROFILE`	Non	Profil AWS local, alternative aux clés
`AWS_S3_ENDPOINT`	Non	Endpoint d'un stockage compatible S3 (Scaleway, OVH, Cloudflare R2, MinIO…)
`AWS_S3_FORCE_PATH_STYLE`	Non	`true` si le fournisseur l'exige (MinIO notamment)
`AWS_PUBLIC_BASE_URL`	Non	URL publique de base des fichiers (CDN/domaine du bucket). Sinon : `https://<bucket>.s3.<region>.amazonaws.com/...`
`ADMIN_EMAIL` / `ADMIN_PASSWORD` / `ADMIN_NAME`	Non	Utilisées par `yarn create-admin` pour créer/réinitialiser un administrateur

Variable	Obligatoire	Rôle
`ALLOW_ADMIN_SIGNUP`	Non	`true` pour autoriser temporairement la route `POST /api/signup` (création d'admin). **Laisser à `false`**
`NEXT_OUTPUT_MODE`	Non	`standalone` pour un build autonome (Docker/VPS)
`NEXT_DIST_DIR`	Non	Dossier de build (défaut `.next`)

---

6. Images, stockage et services externes

Images existantes

Les images déjà uploadées sont stockées sur le **stockage cloud de la plateforme d'origine** et sont référencées **par URL** dans la base (colonnes `image` / `images`, et contenus CMS de `SiteSetting`). Deux domaines sont utilisés :

- `https://abacusai-apps-…-us-west-2.s3.us-west-2.amazonaws.com/…/public/uploads/…` (images uploadées via l'admin) ;
- `https://cdn.abacus.ai/images/…` (images d'illustration initiales, ainsi que l'image de bandeau par défaut codée dans `lib/settings.ts` et `components/hero-section.tsx`).

Ces URLs publiques restent dans la base et continueront de s'afficher tant que ces fichiers restent en ligne, **mais leur pérennité n'est pas garantie** une fois le site quitté. Par précaution, **une copie de toutes ces images (70 fichiers) est fournie** dans `assets/images_existantes/`, avec le fichier `correspondance_urls.csv` (fichier local ↔ URL en base).

Migration recommandée : 1. Copier le contenu de `assets/images_existantes/` dans votre propre bucket (ou dans `code/public/images/`). 2. Remplacer les URLs en base, par exemple : `sql UPDATE "Vehicle" SET images = (SELECT array_agg(replace(u, 'https://ANCIEN_DOMAINE/chemin/', 'https://NOUVEAU_DOMAINE/')) FROM unnest(images) u);` (adapter selon les tables/colonnes et le type des champs ; faire de même pour `Category.image`, `Project.image`, `Promotion.image`, `UsedVehicle.images`, `ConfiguratorModel.image`, `SiteSetting.value`). 3. Mettre à jour les URLs par défaut dans `lib/settings.ts`, `components/hero-section.tsx` et `scripts/seed.ts`.

Nouveaux uploads (à reconfigurer)

Le mécanisme d'upload (`lib/s3.ts`, `lib/aws-config.ts`, route `app/api/upload/presigned`) génère une **URL présignée S3** : le navigateur envoie directement le fichier au bucket. Sur la plateforme d'origine, les identifiants étaient fournis par l'environnement ; il faut désormais **configurer votre propre bucket** :

- renseigner `AWS_REGION`, `AWS_BUCKET_NAME`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` (+ `AWS_S3_ENDPOINT` pour un fournisseur non-AWS) ;
- autoriser la **lecture publique** du préfixe `<AWS_FOLDER_PREFIX>public/uploads/*` (images affichées sur le site) ; les pièces jointes SAV non publiques (`uploads/*`) sont lues via des URLs présignées depuis l'admin ;
- configurer le **CORS** du bucket pour accepter les requêtes `PUT` depuis le domaine du site, par exemple :

```
[{ "AllowedOrigins": ["https://www.alsimat.fr"], "AllowedMethods": ["PUT", "GET"], "AllowedHeaders":
        ["*"], "MaxAgeSeconds": 3000 }]
```

Services externes et SDK utilisés (vérifiés dans le code)

Service / SDK	Utilisation	Action à prévoir
`@aws-sdk/client-s3`, `@aws-sdk/s3-request-presigner`	Upload présigné et lecture des images / pièces jointes SAV	Configurer votre bucket (ci-dessus)
PostgreSQL (via Prisma)	Toutes les données	Fournir votre base (`DATABASE_URL`)
NextAuth (credentials)	Connexion admin	Définir `NEXTAUTH_SECRET` et `NEXTAUTH_URL`
`cdn.abacus.ai` et bucket S3 de la plateforme	Hébergement des images existantes	Migrer les images (copie fournie)
Google Fonts (via `next/font`)	Polices DM Sans, Plus Jakarta Sans, JetBrains Mono	Aucune (téléchargées au build)

Envoi d'emails : aucun service d'envoi d'emails n'est intégré dans le code. Les demandes de devis, tickets SAV et messages de contact sont uniquement enregistrés en base et consultables dans l'admin. Pour recevoir des notifications par email, il faudra ajouter un service (ex. Resend, Brevo, SMTP via Nodemailer) dans les routes `app/api/quotes`, `app/api/sav` et `app/api/contact`.

`@azure/storage-blob` figure dans `package.json` mais n'est pas utilisé par le code (dépendance pouvant être retirée).

Adaptations effectuées pour rendre le code autonome

Par rapport à la version hébergée, la copie fournie a été nettoyée : - suppression du script de la plateforme d'origine chargé dans `app/layout.tsx` et du module de remontée d'erreurs injecté par `next.config.js` ; - `prisma/schema.prisma` : suppression du chemin de sortie absolu et de la cible binaire propre à la plateforme ; - `.yarnrc.yml` simplifié (suppression des chemins de cache de la plateforme) ; - `lib/aws-config.ts` / `lib/s3.ts` : prise en charge des clés explicites, d'un endpoint compatible S3 et d'une URL publique personnalisée ; - **sécurité** : `scripts/seed.ts` contenait des mots de passe administrateurs en clair ; ils ont été retirés et remplacés par les variables `ADMIN_EMAIL` / `ADMIN_PASSWORD` ; - **sécurité** : la route `POST /api/signup` permettait à n'importe qui de créer un compte administrateur ; elle est désormais désactivée sauf si `ALLOW_ADMIN_SIGNUP=true` ; - ajout de `scripts/create-admin.ts` (`yarn create-admin`), de `.env.example` et de `.gitignore`.

Le build de production (`yarn build`) et le démarrage (`yarn start`) de cette copie ont été testés avec la base restaurée.

---

7. Comptes administrateurs

- Le rôle est défini par le champ `role` du modèle `User` (valeur `"admin"`). Le middleware (`middleware.ts`) et la fonction `requireAdmin()` (`lib/admin-auth.ts`) n'autorisent l'accès à `/admin` et aux API `/api/admin/*` qu'aux utilisateurs ayant ce rôle. Les mots de passe sont stockés **hashés (bcrypt)** dans `hashedPassword`.

- Les comptes administrateurs existants **mdupuy@alsimat.fr** et **wd@carlite.fr** sont présents dans la base exportée, avec leurs mots de passe actuels (hashés).

Créer un nouvel administrateur ou réinitialiser un mot de passe :

```
cd code
ADMIN_EMAIL="prenom.nom@alsimat.fr" ADMIN_PASSWORD="UnMotDePasseSolide" ADMIN_NAME="Prénom Nom" yarn creat
        e-admin
```

Si l'email existe, le mot de passe est remplacé et le rôle forcé à `admin` ; sinon le compte est créé. Supprimer un compte : `DELETE FROM "User" WHERE email = '...';` dans `psql`.

---

8. Arborescence du projet

```
alsimat_site/
├── README.md / README.pdf
├── assets/
│   ├── alsimat_logo_official.png        Logo officiel
│   └── images_existantes/               Copie des images en ligne + correspondance_urls.csv
├── database/
│   ├── alsimat_database.sql             Export pg_dump (schéma + données)
│   └── schema.prisma                    Copie du schéma Prisma
└── code/
    ├── .env.example                     Modèle des variables d'environnement
    ├── package.json / yarn.lock         Dépendances et scripts (dev, build, start, create-admin)
    ├── next.config.js                   Configuration Next.js
    ├── tailwind.config.ts / postcss.config.js   Styles (Tailwind)
    ├── tsconfig.json                    Configuration TypeScript (alias "@/")
    ├── middleware.ts                    Protection des routes /admin (rôle admin)
    ├── STYLE_GUIDE.md                   Charte graphique / conventions UI
    ├── app/                             Pages et API (App Router)
    │   ├── layout.tsx, page.tsx         Gabarit global et page d'accueil
    │   ├── gammes/[slug]/               Pages catégories / sous-catégories
    │   ├── vehicules/[slug]/            Fiches véhicules
    │   ├── realisations/, occasion/, promotions/, configurateur/
    │   ├── sav/, contact/, a-propos/
    │   ├── sitemap.ts, robots.ts        SEO
    │   ├── admin/                       Back-office (une page par module + login)
    │   └── api/
    │       ├── admin/*                  API protégées (CRUD des modules, stats, réglages, file-url)
    │       ├── auth/[...nextauth]/      Authentification NextAuth
    │       ├── upload/presigned/        Génération d'URL d'upload S3
    │       ├── quotes/, sav/, contact/  Réception des formulaires publics
    │       ├── vehicles/, categories/, projects/, settings/   Lecture publique
    │       └── signup/                  Création d'admin (désactivée par défaut)
    ├── components/                      Composants (header, footer, sections d'accueil, formulaire de dev
is…)
    │   ├── admin/                       Shell de l'admin, uploader d'images
    │   ├── layouts/                     Gabarits de mise en page
    │   └── ui/                          Composants shadcn/ui
    ├── hooks/                           Hooks React
    ├── lib/
    │   ├── db.ts                        Client Prisma
    │   ├── auth.ts, admin-auth.ts       Configuration NextAuth, contrôle du rôle admin
```

```
│   ├── s3.ts, aws-config.ts         Stockage S3 (upload présigné, URLs)
│   ├── settings.ts                  Réglages du site (visibilité, contenus CMS par défaut)
│   └── slug.ts, types.ts, utils.ts  Utilitaires
├── prisma/schema.prisma             Schéma de la base
├── public/                          Fichiers statiques (brand/alsimat-logo.png, favicon, og-image)
├── scripts/
│   ├── seed.ts, safe-seed.ts        Données initiales pour une base vide
│   └── create-admin.ts              Création / réinitialisation d'un administrateur
└── types/next-auth.d.ts             Typage de la session (id, role)
```

---

9. Conseils pour la suite

- Versionner `code/` dans un dépôt Git privé (le `.gitignore` exclut déjà `.env` et `node_modules`).
- Mettre en place des sauvegardes régulières de la base (`pg_dump`).
- Changer les mots de passe des comptes admin après la reprise (`yarn create-admin`).
- Ne jamais committer le fichier `.env`.